# FedGrid website guide

How each part of the website works, where every number comes from, and how data flows from the raw ICED files to the published page.

## What the website is

The website presents the FedGrid project: federated, graph-aware and explainable 3-hour-ahead electricity load forecasting, run on real hourly demand for the six Southern Region states (Telangana, Andhra Pradesh, Karnataka, Tamil Nadu, Kerala, Puducherry). It is one self-contained HTML page. All charts and tables are drawn in the visitor's browser by JavaScript embedded in the page, using result files produced by the Python pipeline.

**Headline result.** Personalised federated learning with the neighbour feature reaches an RMSE of 487.1 MW across all six states in 2025, 10.5% lower than plain FedAvg (544.6 MW). The neighbour feature itself adds only 0.18% on RMSE and 0.82% on peak-hour MAE.

## The end-to-end flow

Orange boxes are inputs from outside the project. Blue boxes are the pipeline scripts, run in order 1 to 5. Each script writes files that the next one reads.

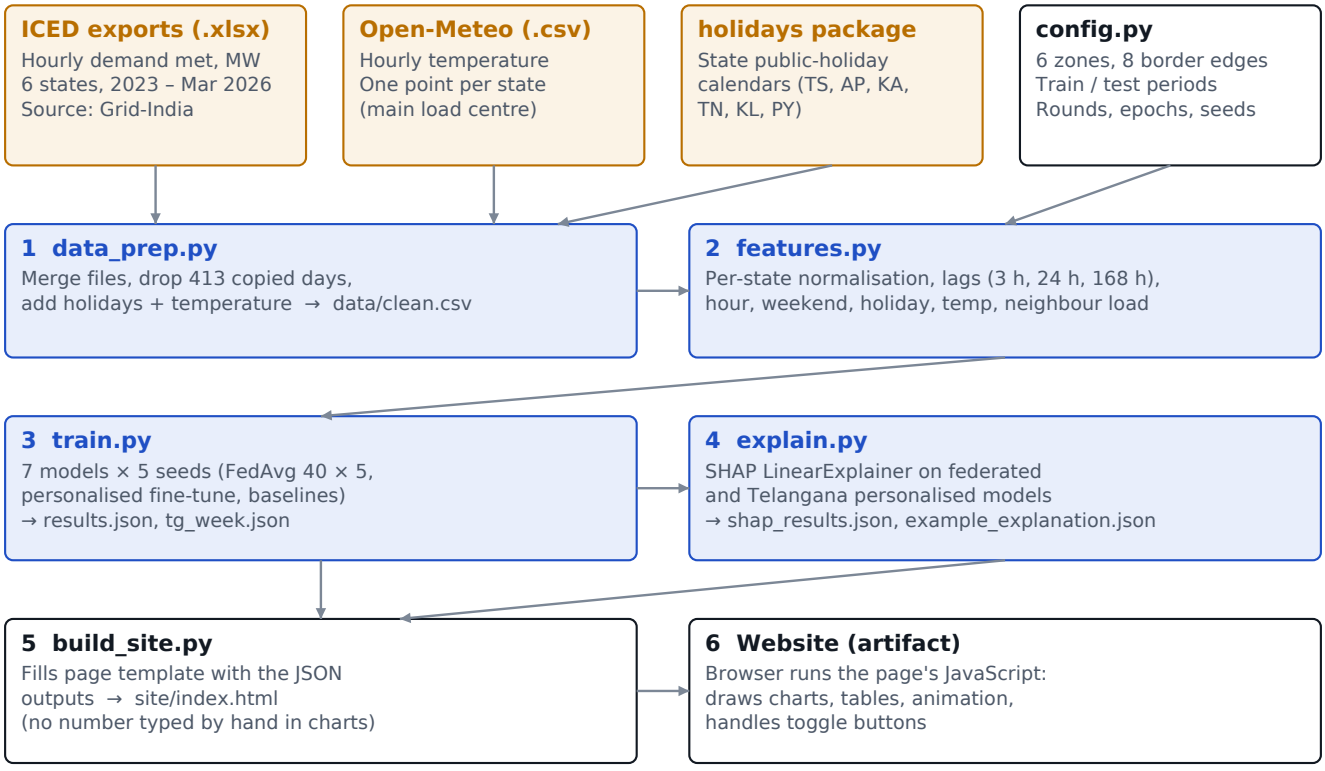



Figure 1. Data and code flow, from source files to the published website.

# Step by step: the pipeline

Run the scripts in this order from the project folder. Each step can be rerun on its own if its inputs have not changed.

## Step 1. data_prep.py: clean the raw data

- Reads the 12 ICED exports in `data/raw/`. Each row has a state-year label (e.g. 'Telangana - 2025'), a date-hour ('01-Jan 12am') and demand met in MW. The script splits the label to recover the year and builds a full timestamp.
- Drops ICED's footer rows (copyright and disclaimer lines).
- Removes copied data. ICED repeats some blocks exactly: April 2024 is a copy of March 2024 and March 2026 is a copy of January 2026 in every state. Any day whose 24 hourly values exactly match an earlier day of the same state is set to blank. 413 days were removed in total.
- Adds `is_holiday` from each state's public-holiday calendar, and `temperature_c` from the Open-Meteo file.
- Writes `data/clean.csv` and `outputs/data_quality.json`. The website's Data table is built from data_quality.json.

## Step 2. features.py: build model inputs

- Each state scales its own load using its own 2023–2024 mean and standard deviation. No state needs another state's raw numbers, which keeps the federated setting honest.
- For every target hour it builds: `lagH` (load 3 hours earlier, the latest value known when the forecast is made), `lag24` (same hour yesterday), `lag168` (same hour last week), `hod_sin`/`hod_cos` (hour of day on a circle, so 11 pm sits next to midnight), `is_weekend`, `is_holiday` and `temperature_c`.
- The graph feature `neighbor_lagH` is the average scaled load of the bordering states, 3 hours before the target hour. Telangana's neighbours are Andhra Pradesh and Karnataka.
- Rows with any missing input (for example right after a removed day) are skipped.

## Step 3. train.py: train and score the models

Seven models are trained on 2023–2024 and tested on every hour of 2025, then checked again on January–February 2026. Each model is run with 5 random seeds and the results are averaged.

| Model | How it is trained | Data sharing |
|---|---|---|
| Centralized, local / graph-aware | All six states' rows pooled into one linear model | Needs everyone's raw data (reference only) |
| FedAvg, local / graph-aware | Each state trains 5 epochs on its own data; server averages the weights, weighted by row count; repeated for 40 rounds | Only model weights leave a state |
| Each state alone | One model per state, its own data only | None |
| Personalised, local / graph-aware | FedAvg as above, then each state fine-tunes the shared model for 5 more epochs on its own data | Only model weights leave a state |

- Models are `SGDRegressor` linear models from scikit-learn. The script was checked against exact closed-form fits; the results agree almost exactly, so training converged.
- Predictions are converted back to MW with each state's own mean and standard deviation before scoring.
- Writes `results.json` (all metrics, per state and pooled, mean and spread over seeds), `fed_graph_model.json` (the global model's weights) and `tg_week.json` (Telangana's peak week).

## Step 4. explain.py: SHAP explanations

- Uses SHAP's LinearExplainer. For a linear model, a feature's SHAP value is its weight times how far its value is from the training average. The values for one forecast add up exactly to the difference between that forecast and the model's average forecast.
- Explains the global federated model across all states, and Telangana's personalised model in MW, both for all hours and for peak hours.
- Builds a worked example for Telangana's highest-demand hour of 2025 (2025-03-20, 4 pm) and saves it to `example_explanation.json`.

## Step 5. build_site.py: assemble the page

- Takes the page template (`site/_head.html` for styles and `site/_body.html` for content and scripts) and pastes the five JSON output files into the page's script.
- Every number in a chart or table is read from those files at view time, so rerunning the pipeline and rebuilding updates the whole site. Only a few figures in the running text (such as −10.5%) are typed into the template and must be updated by hand if results change.

## Commands

```
pip install pandas numpy scikit-learn shap holidays openpyxl
python data_prep.py
python train.py    # about 2 minutes
python explain.py
python build_site.py    # writes site/index.html
```

# Section by section: the website

Each part of the page in the order a visitor sees it: what it shows, where the data comes from, and what the interactive parts do.

## Top navigation bar

| Element | What it shows and how it works |
|---|---|
| Links | Jump to each section of the page: The gap, Data, Method, Results, Telangana, Explainability, Roadmap, Cite. The bar stays fixed at the top while scrolling. |
| FedGrid name | Returns to the top of the page. |

## Opening section (hero)

| Element | What it shows and how it works |
|---|---|
| Paper title | The full title of the manuscript. |
| Summary line | One sentence on what the system does. |
| Data note | States the data source (Grid-India hourly demand) and the period used, January 2023 to February 2026. |
| Grid diagram | Six circles are the six states; grey lines are the 8 real shared borders (TG–AP, TG–KA, AP–KA, AP–TN, KA–TN, KA–KL, TN–KL, TN–PY). The dashed blue lines run to the FedAvg server. The layout roughly follows the map. |
| Animation | Illustrative only. Blue dots travel to the server and back once per round, counting Rounds 1 to 40, to show that only model weights travel. The orange ring in each circle stands for local demand data that never leaves the state; its size follows that state's typical load level. Every few rounds an orange glow spreads from one state to its neighbours to show how demand events can be related across borders. Visitors with reduced-motion settings see a still image. |

## Key figures strip

| Element | What it shows and how it works |
|---|---|
| Error cut | −10.5% RMSE: personalised federated (graph-aware) vs plain FedAvg (graph-aware), 2025, all states. From results.json. |
| Clients, history, horizon | 6 states with 8 borders; over 26,000 usable hourly rows per state after cleaning; forecasts 3 hours ahead. |
| Federation, test year | 40 rounds × 5 local epochs; the whole of 2025 (8,760 hours per state) is held out for testing. |

## 01 The gap (introduction and related work)

| Element | What it shows and how it works |
|---|---|
| Three cards | The project's three properties: federated (raw data stays in the state), graph-aware (neighbours inform the forecast), explainable (every forecast has reasons). |
| Comparison table | Summarises the paper's literature review: which lines of prior work have which of the three properties, with reference numbers. FedGrid is the row with all three. |

## 03 Problem statement

| Element | What it shows and how it works |
|---|---|
| Formal statement | Zones Z linked by graph G = (Z, E); learn one forecasting model without sharing raw records. |
| Goals (a) and (b) | Beat a federated model without the graph, especially in peak hours; and attribute every forecast to its inputs. The results section reports how far goal (a) was met. |

## 04 Data

| Element | What it shows and how it works |
|---|---|
| Text | Explains the source (ICED, Grid-India) and lists the 8 border edges. |
| State table | For each state: average and highest hourly demand met, days removed as copies, and public-holiday days. Built from data_quality.json. Telangana is highlighted. |
| Note | Explains the copied-data problem, the weather source and the holiday source. |

## 05 Method

| Element | What it shows and how it works |
|---|---|
| Three cards | One per script (data_prep.py, train.py, explain.py) with what each does. The arrows show the order. |
| Setup grid | Fixed settings of the experiment: clients and edges, horizon, training and test periods, seeds, metrics, peak-hour metric. |
| Features note | Lists the model inputs and states that observed temperature stands in for a weather forecast. |

## 06 Results

| Element | What it shows and how it works |
|---|---|
| Bar chart | One bar per model (7 models), all six states pooled. The blue bar is FedGrid (personalised, graph-aware). Bars are drawn to scale from zero. |
| Metric buttons | MAE, RMSE, MAPE % or Peak MAE. Clicking one redraws the bars and the three change figures underneath. |
| Period buttons | 2025 (main test year) or Jan–Feb 2026 (an extra check on data newer than anything the models saw). |
| Three change figures | Computed live as (new − old) / old: Personalisation = plain FedAvg → personalised; Graph feature = personalised without → with neighbour feature; vs each state alone = solo models → personalised federated. Blue means lower error, orange means higher. |
| Table 1 | Every model with mean ± standard deviation over the 5 seeds, 2025. |
| Three findings | The written conclusions: plain FedAvg underfits; personalised federation works; the neighbour feature helps only a little. |

## 06 Telangana

| Element | What it shows and how it works |
|---|---|
| Three tiles | Telangana's 2025 MAPE with the personalised model (4.12%), its RMSE change vs plain FedAvg (-18.7%, 681 → 554 MW), and its peak-hour MAE (392 MW, was 492). Computed in the browser from results.json. |
| Week chart | 17–23 March 2025, the week of Telangana's highest 2025 hour (16,756 MW, 4 pm on 20 March, marked with a dot). Black = actual demand, blue = personalised federated forecast, dashed orange = plain FedAvg forecast. All forecasts are made 3 hours ahead. Data from tg_week.json. |
| State table | RMSE per state for plain FedAvg, personalised federated, and each state alone, plus the change from plain to personalised. |

## 07 Explainability

| Element | What it shows and how it works |
|---|---|
| SHAP bar chart | For Telangana's personalised model: the average size (in MW) of each feature's push on the forecast. Blue bar = the neighbour feature, orange = temperature, grey = the state's own features. |
| All hours / Peak hours | Switches between all 2025 hours and only the busiest 10% of hours. In peak hours the load history features push harder and temperature rises. |
| Worked example | One real forecast, 2025-03-20 at 4 pm. It starts from the model's typical forecast (9,548 MW), adds each feature's contribution (blue bars push up, orange push down), and ends at the forecast (16,346 MW) next to the actual value (16,756 MW). The contributions add up exactly to the forecast. |
| Note | Gives the neighbour feature's small average contribution and why a graph neural network is the next step. |

## 08 Discussion and future work

| Element | What it shows and how it works |
|---|---|
| Text | What the real-data results mean for the project. |
| Built / Next lists | What the project has done (blue 'Done' tags) and what is planned (orange 'Planned' tags). |

## Cite this work and footer

| Element | What it shows and how it works |
|---|---|
| Team cards, BibTeX | Author names and a citation entry. The Copy BibTeX button copies it to the clipboard; if the browser blocks copying, it selects the text so it can be copied by hand. |
| Data sources | Links to ICED and Open-Meteo, and the holidays package. |
| References | The paper's 11 references with DOI links. |
| Footer | Data credit, and a note that the grid animation is illustrative. |

# Terms used on the website

| Term | Meaning |
|---|---|
| Demand met | Electricity actually supplied in each hour (MW). It can be lower than true demand during shortages. |
| FedAvg | Federated averaging. Each state trains on its own data, sends only model weights to a server, and the server averages them (states with more rows count more). Repeated for 40 rounds. |
| Personalised | After FedAvg, each state fine-tunes the shared model on its own data for 5 epochs. Each state ends with its own version. |
| Graph / neighbour feature | neighbor_lagH: the average recent load of the states that share a border with this one. |
| Epoch / round | Epoch: one pass over a state's training data. Round: every state trains locally, then the server averages. |
| MAE | Mean absolute error: the average size of the forecast miss, in MW. |
| RMSE | Root mean squared error, in MW. Like MAE but punishes large misses more. |
| MAPE | Mean absolute percentage error: the average miss as a percentage of actual demand. Treats small and large states equally. |
| Peak MAE | MAE over only the busiest 10% of hours for each state. |
| Pooled | All six states' hours scored together. MAE and RMSE in MW are dominated by large states such as Tamil Nadu; MAPE is not. |
| SHAP value | How much one input pushed one forecast up or down, in MW. A forecast equals the baseline plus all its SHAP values. |
| Seed | A random starting point. Running 5 seeds and averaging shows the results are not luck; the ± values are the spread. |

## Limits to keep in mind

- The models are linear stand-ins for the graph neural network named in the paper title.
- Temperature is the recorded value at the target hour, standing in for a forecast. Real use would have slightly larger errors.
- States are large zones. The neighbour feature may matter more at substation or district level.
- The grid animation is an illustration, not a replay of real data.